

Part 1 — Your First Python Program

```
# Part 1 -- Your First Python Program
import socket
print("Socket library loaded successfully")

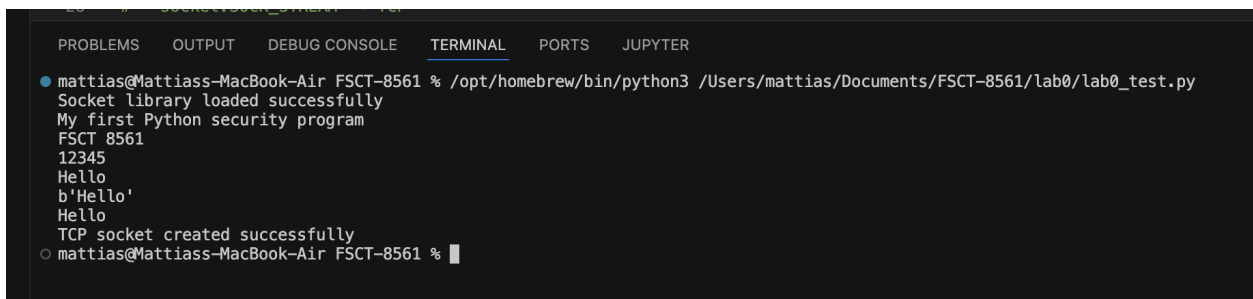
print("My first Python security program")

course = "FSCT 8561"
port = 12345
print(course)
print(port)

message = "Hello"
data = message.encode()
print(message)
print(data)
print(data.decode())
```

Part 2 — Understanding Sockets

```
import socket
my_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
print("TCP socket created successfully")
my_socket.close()
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER
● mattias@Mattiass-MacBook-Air FSCT-8561 % /opt/homebrew/bin/python3 /Users/mattias/Documents/FSCT-8561/lab0/lab0_test.py
Socket library loaded successfully
My first Python security program
FSCT 8561
12345
Hello
b'Hello'
Hello
TCP socket created successfully
○ mattias@Mattiass-MacBook-Air FSCT-8561 %
```

Reflection Question 1

1. What is a socket?

A socket is an endpoint that my program uses to send or receive data over a network, where it is identified by the IP address and port number.

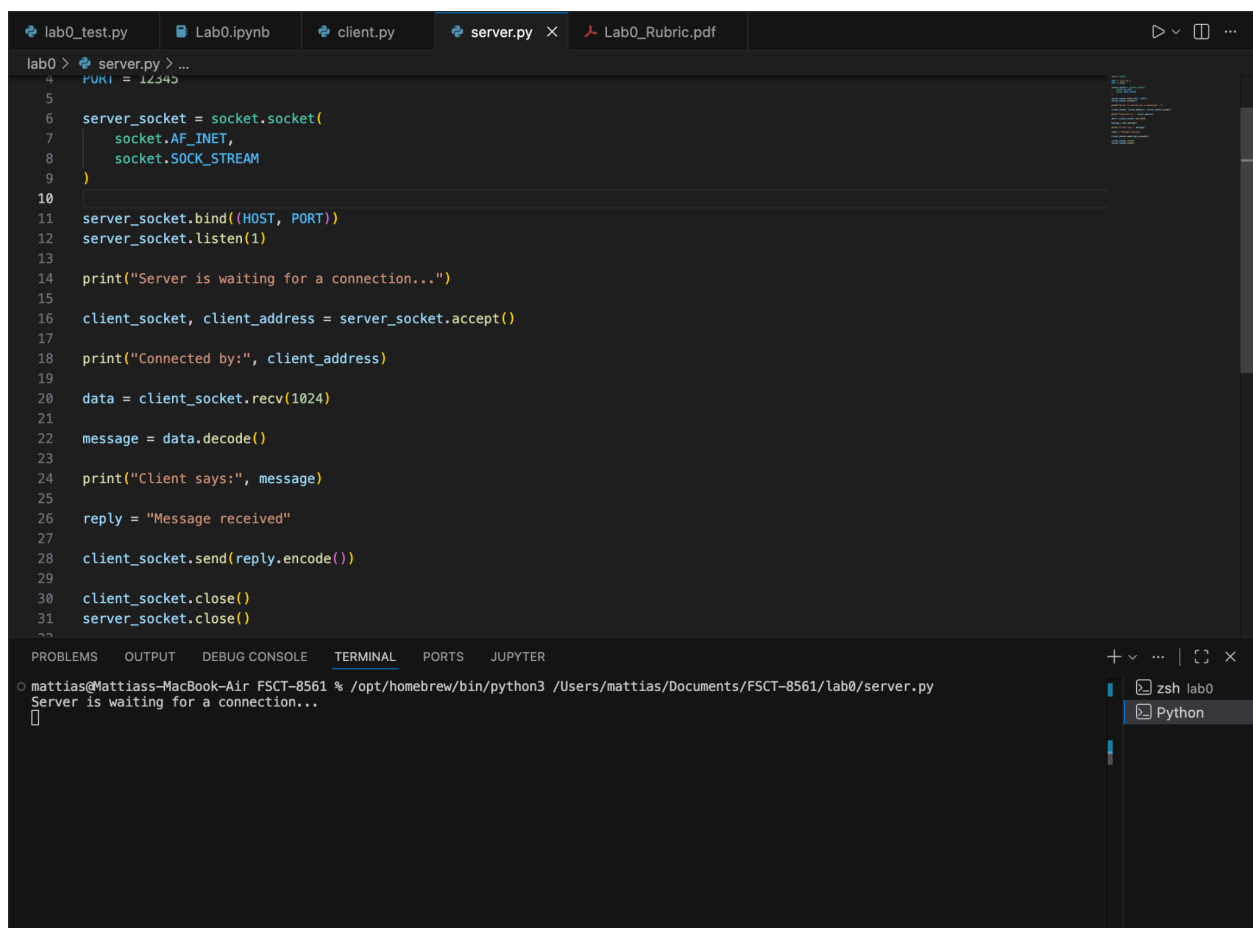
2. What does `AF\_INET` mean?

AF_INET is used to specify the address family for the socket, which in this case is IPv4. It tells us that the socket will use IPv4 addresses.

3. What does `SOCK\_STREAM` mean?

SOCK_STREAM represents, in this case, the socket type, meaning this socket uses TCP

Part 4 — Create a Basic TCP Server



The screenshot shows a JupyterLab environment with a file explorer at the top displaying 'lab0_test.py', 'Lab0.ipynb', 'client.py', 'server.py', and 'Lab0_Rubric.pdf'. The 'server.py' file is open in the editor, showing the following Python code:

```
lab0 > server.py > ...
4  PORT = 12345
5
6  server_socket = socket.socket(
7      socket.AF_INET,
8      socket.SOCK_STREAM
9  )
10
11 server_socket.bind((HOST, PORT))
12 server_socket.listen(1)
13
14 print("Server is waiting for a connection...")
15
16 client_socket, client_address = server_socket.accept()
17
18 print("Connected by:", client_address)
19
20 data = client_socket.recv(1024)
21
22 message = data.decode()
23
24 print("Client says:", message)
25
26 reply = "Message received"
27
28 client_socket.send(reply.encode())
29
30 client_socket.close()
31 server_socket.close()
32
```

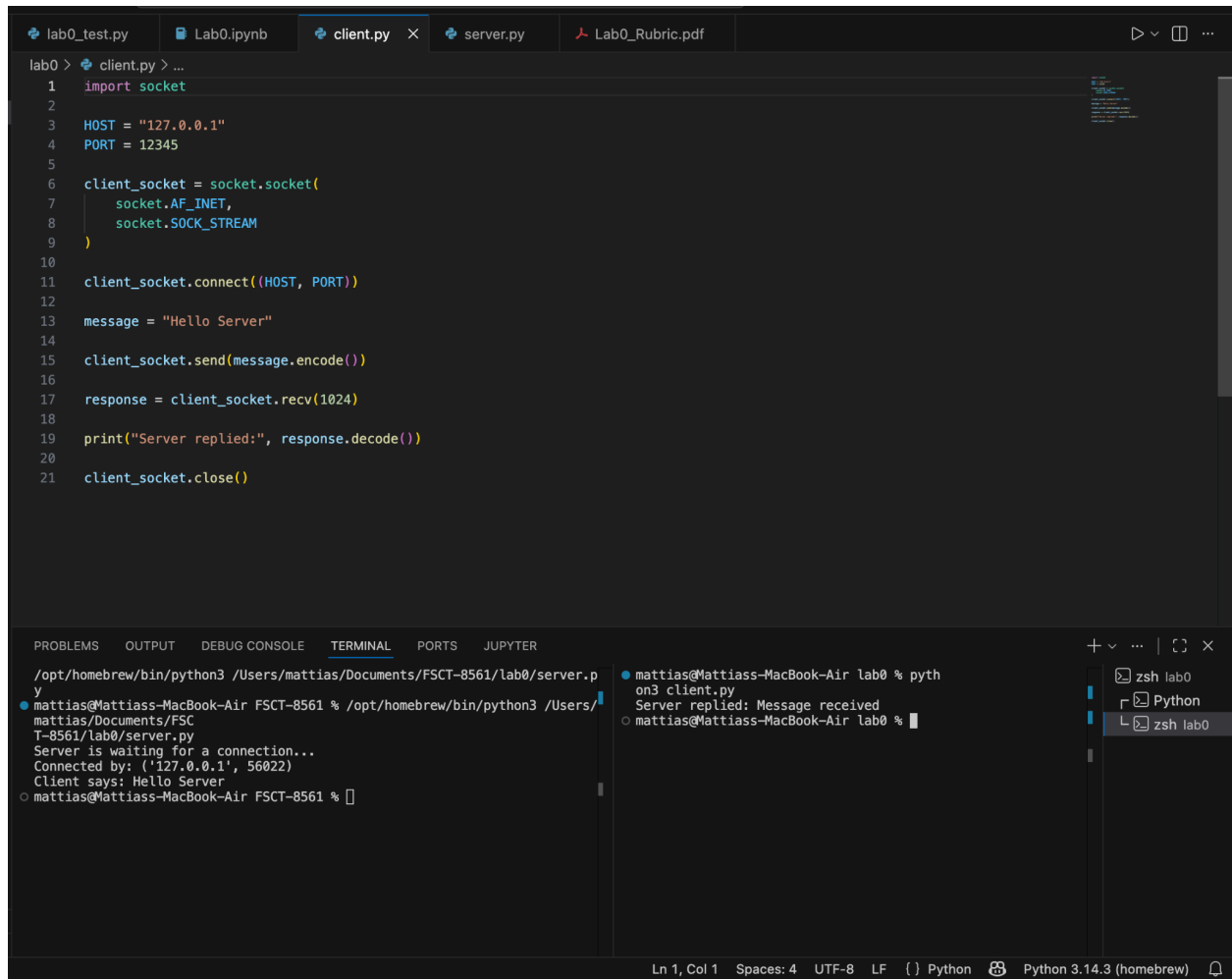
Below the code editor is a terminal window showing the execution of the script:

```
mattias@Mattiass-MacBook-Air FSCT-8561 % /opt/homebrew/bin/python3 /Users/mattias/Documents/FSCT-8561/lab0/server.py
Server is waiting for a connection...

```

On the right side of the terminal, there is a sidebar with tabs for 'zsh lab0' and 'Python'.

Part 5 — Create a Basic TCP Client



The screenshot shows a code editor with a file named `client.py` open. The code is a Python script that creates a TCP client socket, connects to a server at `127.0.0.1` on port `12345`, sends the message "Hello Server", and prints the server's response. The terminal output shows the server script (`server.py`) running and receiving the connection, followed by the client script (`client.py`) running and sending the message. The server's response is "Server replied: Message received".

```
lab0 > client.py > ...
1 import socket
2
3 HOST = "127.0.0.1"
4 PORT = 12345
5
6 client_socket = socket.socket(
7     socket.AF_INET,
8     socket.SOCK_STREAM
9 )
10
11 client_socket.connect((HOST, PORT))
12
13 message = "Hello Server"
14
15 client_socket.send(message.encode())
16
17 response = client_socket.recv(1024)
18
19 print("Server replied:", response.decode())
20
21 client_socket.close()
```

```
/opt/homebrew/bin/python3 /Users/mattias/Documents/FSCT-8561/lab0/server.py
mattias@Mattiass-MacBook-Air FSCT-8561 % /opt/homebrew/bin/python3 /Users/mattias/Documents/FSCT-8561/lab0/server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 56022)
Client says: Hello Server
mattias@Mattiass-MacBook-Air FSCT-8561 %
```

```
mattias@Mattiass-MacBook-Air lab0 % python3 client.py
Server replied: Message received
mattias@Mattiass-MacBook-Air lab0 %
```

Part 6 — Modify the Client and Server

```
# TASK A: message now includes my name
message = "Hello Server, this is Mattias"
```

```
lab0 > client.py > ...
1  import socket
2
3  HOST = "127.0.0.1"
4  PORT = 12345
5
6  client_socket = socket.socket(
7      socket.AF_INET,
8      socket.SOCK_STREAM
9  )
10
11 client_socket.connect((HOST, PORT))
12 # TASK A: message now includes my name
13 message = "Hello Server, this is Mattias"
14
15 client_socket.send(message.encode())
16
17 response = client_socket.recv(1024)
18
19 print("Server replied:", response.decode())
20
21 client_socket.close()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER

```
mattias@Mattiass-MacBook-Air FSCT-8561 % /opt/homebrew/bin/python3
/Users/mattias/Documents/FSCT-8561/lab0/server.py
Connected by: ('127.0.0.1', 56185)
Client says: Hello Server, this is Mattias
mattias@Mattiass-MacBook-Air FSCT-8561 %
```

● mattias@Mattiass-MacBook-Air FSCT-8561 % cd lab0
● mattias@Mattiass-MacBook-Air lab0 % python3 client.py
Server replied: Message received
○ mattias@Mattiass-MacBook-Air lab0 %

zsh lab0
Python
zsh lab0

TASK B: change server reply message
reply = "Thank you have a good day! - Mattias"

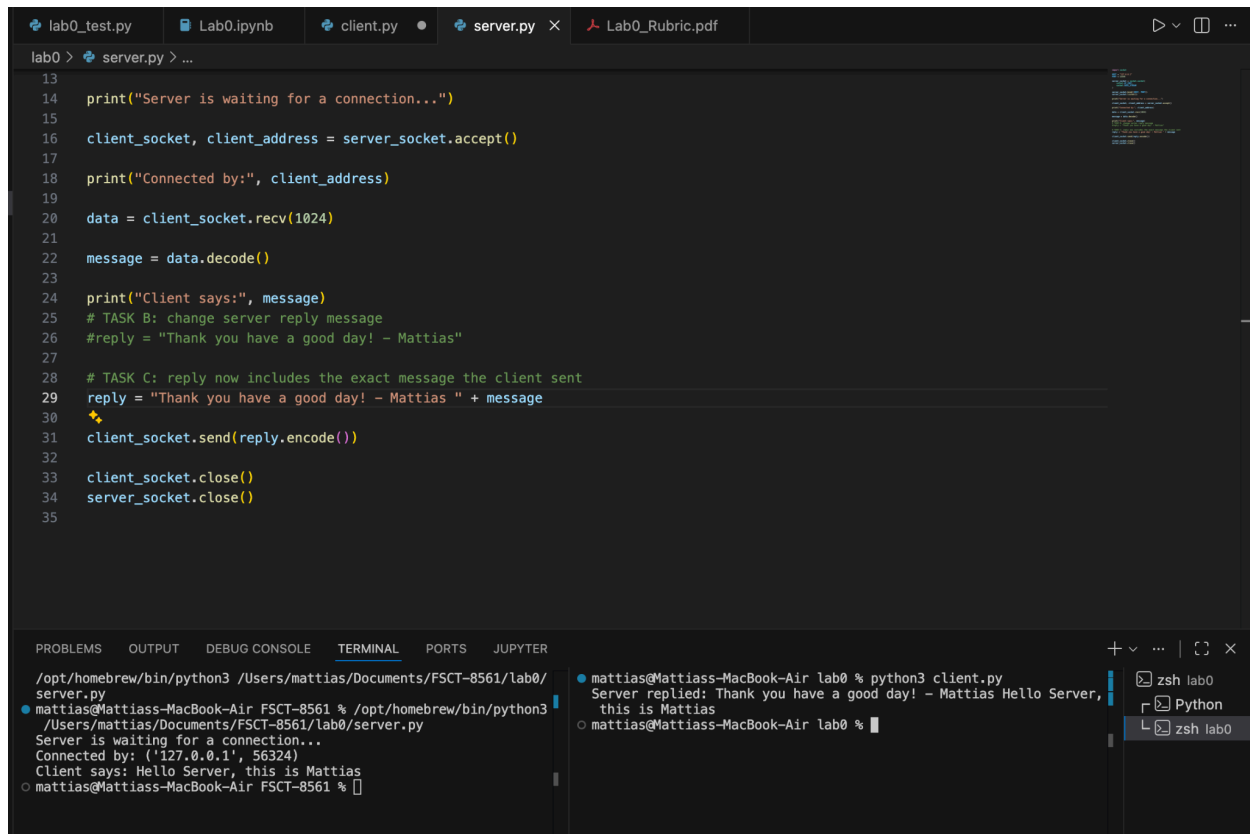
```
22 message = data.decode()
23
24 print("Client says:", message)
25 # TASK B: change server reply message
26 reply = "Thank you have a good day! - Mattias"
27
28 client_socket.send(reply.encode())
29
30 client_socket.close()
31 server_socket.close()
32
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER

```
/opt/homebrew/bin/python3 /Users/mattias/Documents/FSCT-8561/lab0/
server.py
● mattias@Mattiass-MacBook-Air FSCT-8561 % /opt/homebrew/bin/python3
/Users/mattias/Documents/FSCT-8561/lab0/server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 56256)
Client says: Hello Server, this is Mattias
mattias@Mattiass-MacBook-Air FSCT-8561 %
```

● mattias@Mattiass-MacBook-Air FSCT-8561 % cd lab0
● mattias@Mattiass-MacBook-Air lab0 % python3 client.py
Server replied: Thank you have a good day! - Mattias
○ mattias@Mattiass-MacBook-Air lab0 %

```
# TASK C: reply now includes the exact message the client sent
reply = "Thank you have a good day! - Mattias" + message
```



The screenshot displays a JupyterLab environment with a dark theme. At the top, a file browser shows several files: `lab0_test.py`, `Lab0.ipynb`, `client.py`, `server.py`, and `Lab0_Rubric.pdf`. The `server.py` file is open in the main editor, showing a Python script for a simple TCP server. The script includes comments for tasks B and C, and a line that concatenates a fixed response with the received message. The terminal at the bottom shows the execution of the script, with output indicating a successful connection and the transmission of the concatenated response.

```
lab0 > server.py > ...
13
14 print("Server is waiting for a connection...")
15
16 client_socket, client_address = server_socket.accept()
17
18 print("Connected by:", client_address)
19
20 data = client_socket.recv(1024)
21
22 message = data.decode()
23
24 print("Client says:", message)
25 # TASK B: change server reply message
26 #reply = "Thank you have a good day! - Mattias"
27
28 # TASK C: reply now includes the exact message the client sent
29 reply = "Thank you have a good day! - Mattias " + message
30
31 client_socket.send(reply.encode())
32
33 client_socket.close()
34 server_socket.close()
35
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER

```
/opt/homebrew/bin/python3 /Users/mattias/Documents/FSCT-8561/lab0/
server.py
● mattias@Mattiass-MacBook-Air FSCT-8561 % /opt/homebrew/bin/python3
/Users/mattias/Documents/FSCT-8561/lab0/server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 56324)
Client says: Hello Server, this is Mattias
○ mattias@Mattiass-MacBook-Air FSCT-8561 %
```

● mattias@Mattiass-MacBook-Air lab0 % python3 client.py
Server replied: Thank you have a good day! - Mattias Hello Server,
this is Mattias
○ mattias@Mattiass-MacBook-Air lab0 %

zsh lab0
Python
zsh lab0

Part 7 — Connect to a Public Web Server

```
lab0 > http_client.py > ...
1 import socket
2
3 client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
4 client.connect(("example.com", 80))
5
6 request = (
7     "GET / HTTP/1.1\r\n"
8     "Host: example.com\r\n"
9     "Connection: close\r\n"
10    "\r\n"
11 )
12
13 client.send(request.encode())
14
15 response = b""
16 while True:
17     data = client.recv(4096)
18     if not data:
19         break
20     response += data
21
22 print(response.decode(errors="replace"))
23 client.close()
```

```
/opt/homebrew/bin/python3 /Users/mattias/Documents/FSCT-8561/lab0/http_client.py
mattias@mattias-MacBook-Air FSCT-8561 % /opt/homebrew/bin/python3 /Users/mattias/Documents/FSCT-8561/lab0/http_client.py
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 16:47:22 GMT
Content-Type: text/html
Transfer-Encoding: chunked
Connection: close
Server: cloudflare
Last-Modified: Wed, 02 Sep 2026 22:14:26 GMT
Allow: GET, HEAD
Accept-Ranges: bytes
Age: 12900
cf-cache-status: HIT
CF-RAY: a38fea07fe6ec55d-YVR

22f
<!doctype html><html lang="en"><head><title>Example Domain</title><link rel="icon" href="data:,"><meta name="viewport" content="width=device-width, initial-scale=1"><style>body{background:#eee;width:60w;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example Domain</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.</p><p><a href="https://iana.org/domains/example">Learn more</a></p></div></body></html>
```

Reflection Question 2

1. Which host did the client connect to?
example.com

2. Which port was used?
Port 80, the default HTTP port.

3. Why is port 80 relevant?
It is relevant because it's the standard port for unencrypted HTTP web traffic.

4. What did the server send back?
It sent back an HTTP response, headers, and the HTML content of example.com's homepage.

Part 8 - Challenge — Rebuild the Client-Server Application Using Socket.IO

Server_socketio.py

```
from aiohttp import web
import socketio

# Create an async Socket.IO server instance
socket_io = socketio.AsyncServer()

# Create the web application and attach Socket.IO to it
app = web.Application()
socket_io.attach(app)

# HTTP route for the root URL
async def index(request):
    return web.Response(text="Hello world from socketio",
        content_type="text/html")

# Runs whenever a client emits a "message" event
@socket_io.on("message")
def print_message(socket_id, data):
    print("Socket ID:", socket_id)
    print("Data:", data)

# Register the index route
app.router.add_get("/", index)

# Start the server
if __name__ == "__main__":
    web.run_app(app)
```

client_socketio.py

```
import socketio

# Create a Socket.IO client instance
sio = socketio.Client()

# Runs when the connection is established
@sio.event
```

```

def connect():
    print("connection established")

# Runs when the client disconnects
@sio.event
def disconnect():
    print("disconnected from server")

# Connect to the server running on localhost, port 8080
sio.connect("http://localhost:8080")

# sends "message" event with some data triggering print_message() on the
server
sio.emit("message", {"data": "my_data"})

# Keep the client running to listen for events
sio.wait()

```

The screenshot shows a JupyterLab environment with two notebooks open: `server_socketio.py` and `client_socketio.py`. The `server_socketio.py` notebook contains the following code:

```

1 from aiohttp import web
2 import socketio
3
4 # Create an async Socket.IO server instance
5 socket_io = socketio.AsyncServer()
6
7 # Create the web application and attach Socket.IO to it
8 app = web.Application()
9 socket_io.attach(app)
10
11 # HTTP route for the root URL
12 async def index(request):
13     return web.Response(text="Hello world from socketio", content_type="text/html")
14
15 # Runs whenever a client emits a "message" event
16 @socket_io.on("message")
17 def print_message(socket_id, data):
18     print("Socket ID:", socket_id)
19     print("Data:", data)
20
21 # Register the index route
22 app.router.add_get("/", index)
23
24 # Start the server

```

The `client_socketio.py` notebook contains the code shown in the first block of the image. The terminal output for `server_socketio.py` shows the following messages:

```

/Users/mattias/Library/Python/3.9/lib/python/site-packages/urllib3
/_init_.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenS
SL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2
.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn(
===== Running on http://0.0.0.0:8080 =====
(Press CTRL+C to quit)
Socket ID: WQ1MFw309v8SiwHjAAAB
Data: {'data': 'my_data'}

```

The terminal output for `client_socketio.py` shows the following messages:

```

/Users/mattias/Library/Python/3.9/lib/python/site-packages/urllib3
/_init_.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenS
SL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2
.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn(
websocket-client package not installed, only polling transport is
available
connection established
^Cdisconnected from server

```


Part 8 — Security Reflection

1. What could happen if a server accepts connections from anyone?

Since `accept()` has no checks on who connects, any client, including malicious ones, could connect and send harmful or unwanted data to the server.

2. Is the Lab 0 message encrypted?

No. The message is sent using plain `.encode().decode()`, so it travels as readable plaintext with no encryption.

3. Could network traffic be observed by another system?

Yes. Because the data isn't encrypted, a tool like Wireshark could intercept and read the message as it travels over the network.

4. Why should servers validate input?

Without validation, the server blindly trusts whatever it receives, which could allow malformed, oversized, or malicious data to crash or exploit the application.

5. Why might authentication be useful?

It ensures only verified, trusted clients can connect and interact with the server, blocking unauthorized access.

6. Suggest two ways to make this application more secure.

We can encrypt traffic using TLS/SSL so intercepted data can't be read.

We can also require authentication and validate all incoming input before processing it.

Part 9 - Deliverables

<https://github.com/itsmat11/FSCT8561-Labs-Mattias>

A short paragraph explaining what you learned and what was unclear:

This lab taught me the basics of TCP client-server communication in Python, including how sockets are created, bound, and used to send and receive data with `encode()/decode()`. What confused me most was the "Address already in use" error I kept hitting when restarting the server, until I learned it's caused by the OS holding the port open briefly after closing. I also ran into some environment setup issues getting `Socket.IO` working on macOS, but once resolved, seeing the client and server connect in real time made `WebSockets` make a lot more sense.

